LIN-933

Métodos Cuantitativos Para Psicolingüistica

Juan Zamora O.

Fundamentos de R para lingüística de corpus





Cómo funciona esta sesión

La prelectura ya cubrió el detalle. Aquí revisamos lo esencial y resolvemos dudas.

Seis bloques, cada uno con:

- Dos láminas de conceptos
- Una lámina de actividades para trabajar en clase

Ten RStudio abierto. Todo lo que aparece se ejecuta.

Objetos, tipos y asignación

Asignación con `<-`. Los nombres distinguen mayúsculas de minúsculas (***case-sensitive***).

```
frecuencia <- 42
class(frecuencia)
#> [1] "numeric"
```

Tres tipos de datos (***data types***) básicos:

Tipo	Contenido	Ejemplo
`numeric`	Números	42, `3.14`
`character`	Texto entre comillas	`"sustantivo"`
`logical`	Valores de verdad	`TRUE`, `FALSE`

Funciones y argumentos

Los argumentos (***arguments***) se pasan por posición o por nombre. Muchos tienen valor predeterminado (***default***).

```
round(3.14159, digits = 2)
#> [1] 3.14
round(3.14159)          # digits vale 0 por defecto
#> [1] 3
```

Funciones propias:

```
frec_norm <- function(f, tam_corpus, base = 1000) {
  (f / tam_corpus) * base
}
frec_norm(340, 85000)
#> [1] 4
```

Ayuda con `?round`. La sección ***Arguments*** es la que más se consulta.

Actividades 1

1.1 Crea tres objetos: tu apellido (`character`), el año de inicio de tu doctorado (`numeric`) y si tu investigación usa corpus (`logical`). Verifica el tipo de cada uno.

1.2 Escribe una función `ttr()` que reciba número de tipos léxicos (***types***) y de ocurrencias (***tokens***) y devuelva su cociente. Pruébala con 450 tipos y 1200 ocurrencias.

1.3 Consulta `?mean` y anota para qué sirve el argumento `na.rm`.

Creación, coerción e indexación

Un vector es una secuencia de elementos del mismo tipo. Si se mezclan, R convierte todo al más general sin avisar: es la coerción (***coercion***).

```
frec <- c(el = 4521, de = 3890, que = 2103, y = 1987)
frec[3]        # por posición
frec["que"]    # por nombre
frec[-1]       # por exclusión
```

La indexación lógica (***logical indexing***) es la forma central:

```
frec[frec > 2000]
#>   el   de  que
#> 4521 3890 2103
```

Vectorización y ordenamiento

Las operaciones son vectorizadas (***vectorized***): se aplican elemento a elemento sin bucles.

```
(frec_abs / 85000) * 1000       # normaliza todo de una vez
```

Comparación: `==`, `!=`, `>`, `<`, `>=`, `<=`, `%in%` Combinación: `&` (y), `|` (o), `!` (negación)

Dos funciones que se confunden:

- `sort()` devuelve el vector ordenado
- `order()` devuelve los índices que producirían ese orden

Actividades 2

2.1 Crea `pos_freq` con NOUN 1250, VERB 890, ADJ 445, ADV 312, PRON 678. Extrae el tercer elemento, todos menos ADV, y el elemento llamado VERB.

2.2 Obtén con indexación lógica las etiquetas con frecuencia mayor a 500 y cuenta cuántas son usando `sum()`.

2.3 Calcula la frecuencia relativa de cada etiqueta como porcentaje del total, redondeada a dos decimales, y grafícala con `barplot()`.

2.4 Ordena `pos_freq` de mayor a menor y obtén el nombre de la etiqueta más frecuente con `which.max()`.

Variables categóricas y niveles

Un factor representa variables categóricas. Sus categorías son los niveles (***levels***), ordenados alfabéticamente por defecto.

```
pos <- factor(c("NOUN", "VERB", "ADJ", "NOUN", "VERB"))
table(pos)
#>  ADJ NOUN VERB
#>    1    2    2
```

- `table()` conserva niveles con frecuencia cero tras filtrar
- `droplevels()` los elimina si estorban
- El argumento `levels` controla el orden en tablas y gráficos

Factores ordinales: la escala se vuelve código

```
acept <- factor(c("marginal", "aceptable", "inaceptable"),
                levels = c("inaceptable", "marginal", "aceptable"),
                ordered = TRUE)
median(acept)
#> [1] marginal
mean(acept)
#> Warning: argument is not numeric or logical
```

Que la mediana funcione y la media no es la teoría de escalas de medición, verificada por el lenguaje.

Trampa clásica: `as.numeric()` sobre un factor devuelve códigos internos. Usar siempre `as.numeric(as.character(x))`.

Actividades 3

3.1 Crea un factor con pretérito, presente y futuro (seis elementos). Reordena los niveles según la secuencia temporal y compara el `barplot(table(...))` antes y después.

3.2 Construye un factor ordinal con niveles A1 a C2 del MCER para ocho aprendientes. Calcula la mediana y filtra los de nivel B2 o superior.

3.3 Crea `puntos <- factor(c(5, 8, 3, 8, 5))`, aplica `as.numeric()` directamente, observa el error y corrígelo hasta que `sum()` devuelva 29.

Estructura e inspección

Un marco de datos (***data frame***) es una tabla: columnas de igual longitud, cada una de un tipo posiblemente distinto.

```
meta <- data.frame(
  texto  = c("T01", "T02", "T03"),
  genero = c("narrativo", "académico", "narrativo"),
  tokens = c(1250, 3400, 890)
)
str(meta)
```

Al abrir cualquier conjunto de datos, ejecutar siempre `str()`. Complementan `dim()`, `head()` y `summary()`.

Filtrado, columnas y agregación

```
meta[meta$tokens > 2000, ]        # indexación lógica
subset(meta, genero == "narrativo")

meta$ttr <- round(meta$types / meta$tokens, 3)   # columna nueva
meta[order(-meta$tokens), ]                      # ordenar

aggregate(tokens ~ genero, data = meta, FUN = mean)
```

La notación y ~ x se lee “y en función de x” y reaparecerá en los modelos estadísticos.

Guardar: `write.csv(..., row.names = FALSE, fileEncoding = "UTF-8")`

Actividades 4

4.1 Crea un marco de datos `hablantes` con seis filas y columnas id, edad, sexo, ciudad y sujeto_nulo. Inspecciónalo con `str()`.

4.2 Filtra los menores de 40 años, primero con indexación lógica y luego con `subset()`.

4.3 Añade una columna `grupo_etario` que valga “joven” bajo los 40 años y “mayor” en caso contrario, usando `ifelse()`.

4.4 Calcula la media de sujeto_nulo por ciudad con `aggregate()`. Guarda el marco como CSV, recárgalo y verifica con `str()` que los tipos se conservaron.

Contenedores heterogéneos

Una lista aloja objetos de cualquier tipo y longitud, incluidos marcos de datos completos.

Distinción central:

- `[ ]` devuelve una **sublista**
- `[[ ]]` devuelve el **contenido** del elemento
- `$nombre` equivale al doble corchete

```
analisis$frecuencias["que"]        # indexación encadenada
```

Por qué importan

Los resultados de las pruebas estadísticas son listas:

```
resultado <- t.test(grupo_a, grupo_b)
names(resultado)
resultado$p.value
```

Extraer `$p.value` directamente permite automatizar reportes sin copiar números a mano.

Aplicar funciones sin bucles:

```
sapply(textos, length)     # simplifica a vector
lapply(textos, length)     # devuelve lista
```

Actividades 5

5.1 Ejecuta un `t.test()` entre dos vectores numéricos inventados y extrae de la lista resultante el valor p (***p-value***) y el estadístico t.

5.2 Escribe una función `describir()` que reciba un vector numérico y devuelva una lista con media, mediana y número de elementos.

5.3 Crea una lista con cuatro vectores de palabras de distinta longitud y obtén con `sapply()` el número de ocurrencias de cada uno.

Condicionales y bucles

`if` evalúa **un solo valor**. Sobre vectores produce error.

Para condiciones elemento a elemento:

```
ifelse(frec > 100, "alta", "baja")            # dos categorías
cut(frec, breaks = c(0, 50, 300, Inf),
    labels = c("baja", "media", "alta"))      # devuelve un factor
```

Bucle (***loop***) `for`: preasignar el contenedor y recorrer con `seq_along()`.

Regla general: si existe versión vectorizada, úsala. `nchar(palabras)` es mejor que un bucle.

Búsqueda de patrones

Instrumento central de la lingüística de corpus:

```
grepl("gat", corpus)                    # devuelve lógicos
grep("gat", corpus)                     # devuelve posiciones
grep("gat", corpus, value = TRUE)       # devuelve los elementos
```

Ambas aceptan expresiones regulares (***regular expressions***) y `ignore.case = TRUE`.

Tres hábitos: comentar el porqué y no el qué, usar nombres descriptivos, fijar la semilla (***seed***) con `set.seed()` siempre que haya azar.

Actividades 6

6.1 Escribe `evaluar_ttr()` que devuelva “alta” sobre 0.5, “media” entre 0.3 y 0.5, y “baja” bajo 0.3. Aplícala a `c(0.62, 0.41, 0.22)` con `sapply()`.

6.2 Dado `frec <- c(5, 120, 45, 890, 12, 340)`, clasifica con `cut()` en tres niveles: baja (hasta 50), media (51 a 300), alta (sobre 300). Verifica que el resultado es un factor.

6.3 Con el vector `corpus`, obtén las oraciones que contienen alguna palabra que empiece por “gat” y cuenta cuántas son.

6.4 Escribe `perfil()` que reciba un vector de oraciones y devuelva una lista con número de oraciones, total de ocurrencias y longitud media por oración.

Qué deberías poder hacer

- Crear objetos, consultar su tipo y escribir funciones propias
- Indexar vectores por posición, nombre y condición lógica
- Reordenar niveles de un factor y declararlo ordinal
- Filtrar marcos de datos y agregar por grupos
- Extraer componentes de una lista de resultados
- Buscar patrones en texto con `grep()`

Si algo quedó en duda, es el momento de decirlo.

Lo que viene

La próxima sesión abre con datos reales, no con sintaxis.

Todo lo de hoy será herramienta, no contenido.

Trae tu script de soluciones y las dudas anotadas.